AMLNN Toolkit Python API Guide

Revision: 0.1

Release Date: 2026-05-15

Copyright

© 2026 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

Amlogic, and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision history

Version	Change Description
V0.1 (2026-05-15)	This is the First Version.

Contents

1 AMLNN API Introduction

2 API Description

2.1 AMLNN Initialization

2.2 Model loading

2.2.1 load_onnx

2.2.2 load_pytorch

2.2.3 load_pt2

2.2.4 load_tflite

2.3 config

2.4 compile

2.5 export_adla

2.6 init_runtime

2.7 get_sdk_version

2.8 load_model

2.9 get_tensor_info

2.10 inference

2.11 get_perf_info

2.12 perf_visualize

2.13 uninit

1 AMLNN API Introduction

This document introduces the APIs and parameter descriptions of the amlnn-toolkit (amlnn-edge-toolkit). Supported Platforms:

- C308L / C302X
- S928X
- A311D2
- T968D4
- S905X5 / S905D5
- C302X2
- C305X2/A123X
- A311Y3

2 API Description

2.1 AMLNN Initialization

When using the API interfaces of amlnn-toolkit, it is necessary to first call `AMLNN()` to initialize the AMLNN object, and call the `uninit()` method to release it upon completion.

Parameter	Type	Description	Required
log_level	str	Set the log level. **Default: 'INFO'**. Currently supported: ['DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL'].	NO

```
#Example

from amlnn.api import AMLNN

amlnn =  AMLNN(log_level='INFO')

...

amlnn.uninit()
```

2.2 Model loading

amlnn_toolkit supports the loading and conversion of models such as ONNX, TFLite (float/quant), and PyTorch (pt/pt2). Different frameworks require calling corresponding interfaces, with detailed explanations provided below.

2.2.1 load_onnx

Function: load onnx model

Return Value: 0: Success, -1: Failure

Parameter	Type	Description	Required
model	str	onnx model file path	YES
inputs	list[str]	Input nodes (tensors) of the ONNX model. Only set when specifying input nodes and shapes is required. **Default value: None.** For multiple inputs, refer to this format: `['input1', 'input2']` Note: **This must be set when the original ONNX input shape is dynamic, or when adjusting the shape dimensions of the original ONNX model is necessary.***	NO
input_shapes	list[list]	Input dimensions of the ONNX model. Only set when specifying input nodes and shapes is required. **Default value: None.** For multiple inputs, refer to this format: `[[1, 3, 224, 224], [1, 3, 229, 229]]` Note: **This must be set when the original ONNX input shape is dynamic, or when adjusting the shape dimensions of the original ONNX model is necessary.**	NO
outputs	list[str]	Specify the output node names (tensor names) for the ONNX model. **Set only when truncating the ONNX model.** Default value: None, indicating no truncation will be performed.	NO

```
#Example1:
amlnn.load_onnx(model='./yolov8s_sim.onnx')

#Example2: Load model and perform truncation.
amlnn.load_onnx(model='./yolov8s_sim.onnx', \
        outputs=
['/model.22/Concat_output_0','/model.22/Concat_1_output_0','/model.22/Concat_2_output_0'])
```

2.2.2 load_pytorch

Function: Load pytorch jit model

Return Value: 0: Success, -1: Failure

Parameter	Type	Description	Required

Parameter	Type	Description	Required
model	str	Path to the PyTorch model file. This refers to the .pt file saved via torch.jit.trace.	YES
inputs	list[str]	Input node names (tensor names) for the PyTorch model. PT models generally do not include input shape information, so external specification is required. For multiple inputs, refer to this format: `['input1', 'input2']`.	YES
input_shapes	list[list]	Input dimensions for the PyTorch model. For multiple inputs, refer to this format: `[[1, 3, 224, 224], [1, 3, 229, 229]]`.	YES
dtypes	list[str]	Model input node data types. Default value: None. When using the default, all inputs are set to float32 by default. For multiple inputs, refer to this format: `['float32', 'float32', 'int32']` Note: **The `dtypes` parameter is optional only when all model input nodes are of type float; otherwise, the input node types must be specified via `dtypes`.**	NO

```
#Example:
amlnn.load_pytorch(model='resnet18.pt',
                inputs=['%x.2'],
                input_shapes=[[1,3,224,224]])
```

2.2.3 load_pt2

Function: Load PyTorch PT2 model. **(Note: This interface is currently not supported in the ARM64 version.)**

Return Value: 0: Success, -1: Failure

Parameter	Type	Description	Required
model	str	Path to the PyTorch model file. This refers to the .pt2 file saved via torch.export.save.	YES
outputs	list[str]	Output node names (tensor names) for the pt2 model. Only set when truncating the pt2 model. **Default value: None, indicating no truncation.**	NO

```
#Example:
amlnn.load_pt2(model='yolov8n_exported.pt2')
```

2.2.4 load_tflite

Function: Load TFLite model.

Return Value: 0: Success, -1: Failure

Parameter	Type	Description	Required
model	str	Path to the TFLite model file.	YES
quantized_model	bool	Specify whether the model is a quantized model. **Default value: False**. When the current TFLite model is quantized, it must be set to True.	NO

```
#Example1:
amlnn.load_tflite(model='yolov8n_sim.tflite')

#Example2:
amlnn.load_tflite(model='yolov8n_sim.tflite',quantized_model=True)
```

2.3 config

Function: Configure model conversion parameters.

Return Value: 0: Success, -1: Failure

Parameter	Type	Description	Required
normalization_mean	[list[float]]	Mean value for each input channel. **Default: None**, indicating that all means are 0.	NO
normalization_std	[list[float]]	Input normalization value. **Default: None**, indicating that the standard deviation is 1.	NO
quantized_dtype	str	Quantization type. **Default value: 'w8a8'**. Currently supported quantization types include: ['w4a8', 'w8a8', 'w4a16', 'w8a16', 'w16a16'].	NO
activation_dtype	str	Specify the activation type. **Default value: 'f16'**. Currently supports ['f16', 'i16','bf16'].	NO
quantized_method	str	Specify the quantization method. **Default value: 'perchannel'**. Currently supports ['perlayer', 'perchannel'].	NO
group_size	int	Sub-channel-wise quantization involves dividing channels into smaller subgroups to achieve finer-grained precision control. **Default value: 32**, currently supports: [32, 64, 128] Note: Only effective for fully connected (FC) layers when `quantized_dtype` is set to w4a8 or w4a16.	NO
activation_quant_algo	str	Specify the activation quantization algorithm. **Default value: 'normal'**, which indicates the use of min-max quantization. Supported values: ['normal', 'histogram', 'percentile', 'omse'].	NO

Parameter	Type	Description	Required
weight_quant_algo	str	Weight quantization algorithm. **Default value: 'normal'**, using min-max quantization. Supported: ['normal', 'decoupleQ','lsq'].	NO
histogram_bins	int	Number of bins for the histogram when the activation quantization algorithm is set to 'histogram'. **Default value: 2048**. Valid values: [1024, 2048, 4096].	NO
omse_range	float	When the activation quantization algorithm is 'omse', the search range. **Default value: 90**. Valid range: (0 to 100).	NO
omse_steps	float	When the activation quantization algorithm is 'omse', the search step size. **Default value: 0.02**. Valid range: (0 to 1).	NO
percentile_sigma	float	When the activation quantization algorithm is 'percentile', the sigma parameter. **Default value: 0.01**. Valid range: (0 to 1).	NO
percentile_alpha	float	When the activation quantization algorithm is 'percentile', the alpha parameter. **Default value: 0.99999**. Valid range: (0 to 1).	NO
hybrid_ratio	float	Hybrid quantization percentage. When quantizing w4a8 and w4a16, control the ratio of 4-bit quantization of convolution. **Default value: 0.5**. Valid range: (0 to 1).	NO
apply_smooth_o3	bool	Specify whether to use the smooth algorithm. **Default value: False**. Valid values: [True, False].	NO
inference_input_type	str	Specifies the input type for the exported ADLA. **Default value: "quant" (consistent with the transformation type)**. When quantized_type is 'w4a8' or 'w8a8', it supports setting ['int8', 'uint8', 'float32']. When quantized_type='w8a16' and activation_dtype='i16', it supports setting ['int16', 'float32'].	NO
inference_output_type	str	Specifies the output type for the exported ADLA. **Default value: "quant" (consistent with the transformation type)**. When quantized_type is 'w4a8' or 'w8a8', it supports setting ['int8', 'uint8', 'float32']. When quantized_type='w8a16' and activation_dtype='i16', it supports setting ['int16', 'float32'].	NO
export_intermediate	bool	Specify whether to export the intermediate IR as a TFLite model file. **Default value: False**.	NO
output_dir	str	Output path. **Default value: None**, meaning the default output path is: model_name + "_adla_unify".	NO
target_platform	str	Specify the platform type. Currently supported: ["PRODUCT_PID0XA001", "PRODUCT_PID0XA002", "PRODUCT_PID0XA003", "PRODUCT_PID0XA004", "PRODUCT_PID0XA005","PRODUCT_PID0XA006", "PRODUCT_PID0XA007","PRODUCT_PID0XA008"]	YES

```
#Example
amlnn.config(target_platform='PRODUCT_PID0XA007')
```

2.4 compile

Function: Compile AMLNN model.

Return Value: 0: Success, -1: Failure

Parameter	Type	Description	Required
dataset	str	Quantization calibration dataset. Set the path to the dataset.txt file (which contains paths to images or .npy files). **Default value: None**, indicating that random data will be used for quantization. This is only for performance testing and does not guarantee accuracy. It is generally recommended to provide 200 to 500 images or npy datasets. Place file path information in dataset.txt. Specific examples are as follows: Single-input model dataset.txt (both image and .npy files are acceptable): 1.jpg 2.jgp Multi-input model dataset.txt (place each set of inputs on a single line, separated by spaces): input1_1.jpg input2_1.jpg input1_2.jpg input2_2.jpg or input1_1.npy input2_1.npy input1_2.npy input2_2.npy Note: When using .npy files as quantization data, the `mean` and `std` parameters configured in the config will not be applied. Therefore, preprocessing must be completed before saving as .npy files, and the shape of the .npy files must match the shape set in the `input_shapes` parameter.	NO
iterations	int	Specify the number of data samples (images or .npy files) to be used for quantization, read from `dataset.txt`. **Default value: -1**, which means iterating through all data. When `dataset` is `None`, only one set of random data will be generated for quantization.	NO

```
#Example
amlnn.compile(dataset='dataset.txt', iterations=200)
```

2.5 export_adla

Function: Export the ADLA model file.

Return Value: 0: Success, -1: Failure

```
#Example
amlnn.export_adla()
```

2.6 init_runtime

Function: Initialize the runtime environment.

Return Value: 0: Success, -1: Failure

Parameter	Type	Description	Required
mode	str	Execution mode. **Default value: 'nnserver'**. Currently supports ['native', 'nnserver'].	NO
board_work_path	str	The device-side working directory, with a default value of `/data/nn`.	NO
enable_perf	bool	Whether to enable performance collection. **Default value: False**. Must be set to `True` if performance information is required.	NO

```
Example
ret = amlnn.init_runtime()
```

2.7 get_sdk_version

Function: Retrieve version information related to the current device-side SDK / ADLA / DELEGATE.

Return value: `str`, version string.

```
#Example:
sdk_version = amlnn.get_sdk_version()
```

2.8 load_model

Function: Load the runtime execution model.

Return Value: 0: Success, -1: Failure

Parameter	Type	Description	Required
path	str	Local model path. Currently only `.adla` files are supported. Note：After completing model conversion and executing `amlnn.init_runtime`, there is no need to explicitly call `load_model` again	YES

```
#Example
amlnn.load_model(path="./adla/resnet18-v1-7_int8_A311D2.adla")
```

2.9 get_tensor_info

Function: Retrieve input and output tensor information of the model.

Return value: dict, containing the following two fields:

- `inputs`: Model input tensor information list.
- `outputs`: Model output tensor information list.

```
#Examples:
tensor_info = amlnn.get_tensor_info()
```

Each tensor's information is represented as a `dict`, with the field descriptions as follows:

Field name	Type	Description
index	int	The index of the current tensor in the input or output list.
name	str	tensor name
n_dims	int	Number of tensor dimensions.
dims	tuple	The native shape of the tensor, corresponding in order to `format_name`.
n_elems	int	Total number of tensor elements.
size	int	The original effective data size of the tensor, in bytes.
format	int	Internal enumeration value of the tensor layout format.
format_name	str	Tensor layout format , such as `NHWC` / `NCHW`
type	int	Internal enumeration value of the tensor data type.
type_name	str	Tensor data type name, such as `INT8`, `UINT8`, `FP32`, etc.
zp	int	Quantization zero point.
scale	float	Quantization scale

2.10 inference

Function: Execute model inference.

Return value: Returns a list of output tensors.

- `list[np.ndarray]`
- The returned output tensor data is in `float32` format.

Parameter	Type	Description	Required
inputs	list/tuple/np.ndarray	Input data, which can be either a single `numpy.ndarray` or a `list` / `tuple`.	YES
inputs_data_format	str	Input data format, supports `NHWC` / `NCHW`, default value is `NHWC`.	NO
outputs_data_format	str	Output data format, supports `NHWC` / `NCHW`, default value is `NHWC`.	NO
run_cycles	str	Number of inference cycles, default value is `1`.	NO

```
#Example
outputs = amlnn.inference(
    inputs=[input_data],
    inputs_data_format="NHWC",
    outputs_data_format="NHWC",
    run_cycles=1,
)
```

2.11 get_perf_info

Function: Retrieve the performance information corresponding to the most recent inference of the current model.

Return value: `str`, a formatted performance information string, including the following fields:

- `npu_avg_time_ms`
- `fps`
- `memory_usage_mb`
- `bandwidth_mb`
- `elapsed_in_hw_op_ms`
- `elapsed_in_sw_op_ms`

Prerequisites for using this interface:

- `enable_perf=True` must be set when calling `init_runtime()`.
- `get_perf_info()` must be called only after a successful call to `inference()`

```
#Example
perf_info = amlnn.get_perf_info()
print(perf_info)

#Output result:
#npu_avg_time_ms=9.624, fps=103.907, memory_usage_mb=5.180, bandwidth_mb=28.802,
elapsed_in_hw_op_ms=5.499, elapsed_in_sw_op_ms=0.000
```

Field name	Description
npu_avg_time_ms	Average NPU inference time in milliseconds; when `run_cycles > 1`, the average across multiple cycles is returned
fps	Frame rate calculated based on `npu_avg_time_ms`
memory_usage_mb	Memory usage required for the current model inference, in MB.
bandwidth_mb	Total bandwidth read/write volume involved in the current model inference, in MB.
elapsed_in_hw_op_ms	Total hardware operator execution time, in milliseconds.
elapsed_in_sw_op_ms	Total software operator execution time, in milliseconds.

2.12 perf_visualize

Function: Generate a visual HTML report based on the performance details obtained from the most recent run.

Visualization Result: Upon successful execution, a folder with the same name as the model will be created in the current working directory, containing the following files:

- `profile_info.txt`
- `hard_op_chart.html`
- `soft_op_chart.html`
- `dram_rd_chart.html`
- `dram_wr_chart.html`
- `pie_charts_distribution.html`

Prerequisites for this interface:

- `perf_visualize()` must be called only after a successful call to `get_perf_info()`.

```
#Example
amlnn.perf_visualize()
```

File Name	Description
profile_info.txt	Original performance details text, containing layer-by-layer performance data, serves as the input file for generating various visual charts.
hard_op_chart.html	Hardware operator time consumption chart, designed to analyze the time distribution across layers on the NPU side.
soft_op_chart.html	Software operator time consumption chart, designed to analyze the time distribution on the CPU side.
dram_rd_chart.html	DRAM read bandwidth chart per layer.
dram_wr_chart.html	DRAM write bandwidth chart per layer.

File Name	Description
pie_charts_distribution.html	Statistical pie charts, displaying the overall distribution of time consumption and bandwidth.

2.13 uninit

Function: Release runtime and model resources.

```
#Example
amlnn.uninit()
```